ENTERPRISE JAVA
(25MCA202)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte

Unit 2

Object Oriented fundamentals and Exception Handling

Inheritance

- Inheritance is a mechanism in which one object acquires all the properties and behaviours of a parent object.
- The idea behind inheritance is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.
- Inheritance represents the *IS-A relationship* which is also known as a *parent-child relationship*.

Why use inheritance

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Terms used in Inheritance

- Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- Super Class/Parent Class: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- Reusability: is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

Inheritance

Syntax

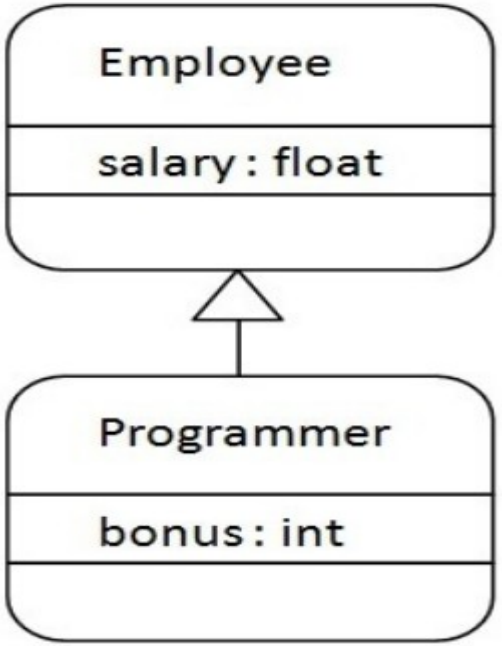
class Subclass-name extends Superclass-name
{
* //methods and fields*
}

```
 class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
 public static void main(String args[]){
   Programmer p=new Programmer();
   System.out.println("Programmer salary is:"+p.salary);
   System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

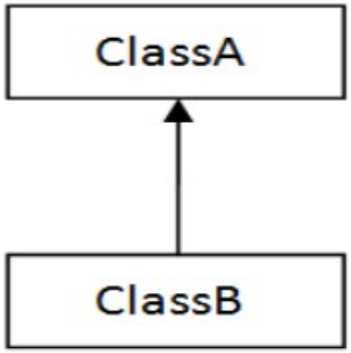
Output:

```
Programmer salary is:40000.0
Bonus of Programmer is:10000
```

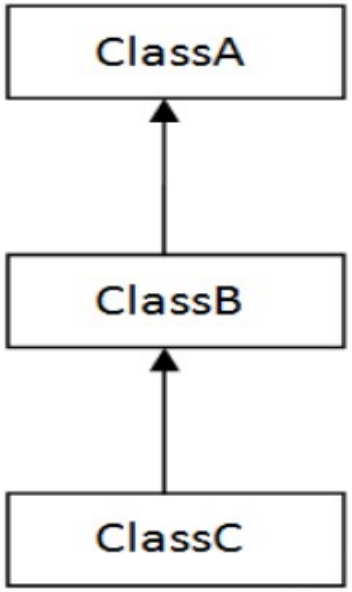
Java Inheritance Example



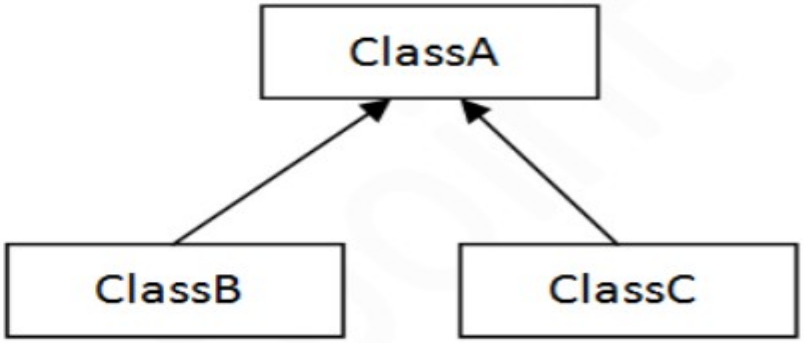
Types of inheritance



1) Single



2) Multilevel



3) Hierarchical

Inheritance

Example

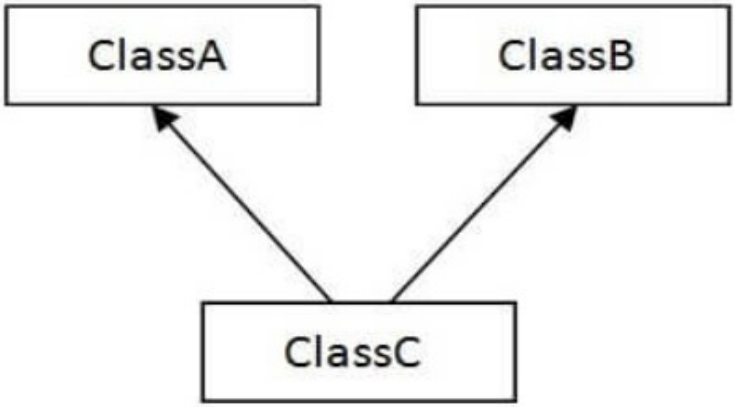
```
class Animal{
void eat(){System.out.println("Animal is eating...");}
}

class Dog extends Animal{
void bark(){System.out.println("Dog is barking...");}
}

class Cat extends Animal{
void meow(){System.out.println("Cat is meowing...");}
}

class Main{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
Dog d=new Dog();
d.bark();
//d.meow();//C.T.Error
}}
```

Output:

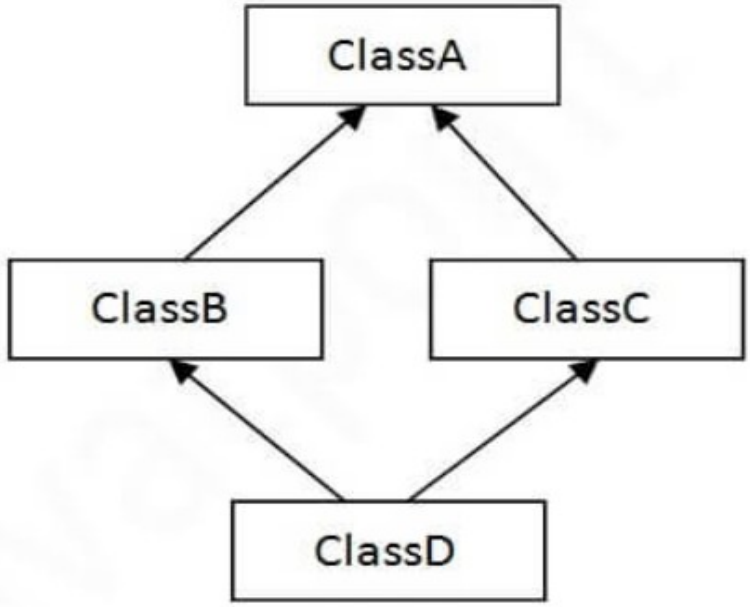
```
Cat is meowing...
Animal is eating...
Dog is barking...
```

Types of inheritance

Note: Multiple and Hybrid inheritance is supported through interface only



4) Multiple



5) Hybrid

Q) Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

```
class A{
void msg(){System.out.println("Hello");}
}
class B{
void msg(){System.out.println("Welcome");}
}

class C extends A,B{ //suppose if it were

 public static void main(String args[]){
   C obj=new C();
   obj.msg();//Now which msg() method would be invoked?
}
}
```

Output:

```
Main.java:8: error: '{' expected
    class C extends A,B{ //suppose if it were
                       ^
1 error
```

Super Keyword

- The super keyword in Java is a reference variable which is used to refer immediate parent class object.
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

- super can be used to refer immediate parent class instance variable.
- super can be used to invoke immediate parent class method.
- super() can be used to invoke immediate parent class constructor.

1) super is used to refer immediate parent class instance variable.

```
class Animal{
String color="white";
}

class Dog extends Animal{
String color="black";
void printColor()
{
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
}

class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

Output:

```
black
white
```

2) super can be used to invoke parent class method

```
class Animal{
void eat(){System.out.println("eating...");}
}

class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark(){System.out.println("barking...");}
void work()
{
 super.eat();
 bark();
}
}

class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

Output:

```
eating...
barking...
```

3) super is used to invoke parent class constructor.

```
class Animal{
Animal(){System.out.println("animal is created");}
}

class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}

class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```

Output:

```
animal is created
dog is created
```

Method Overriding

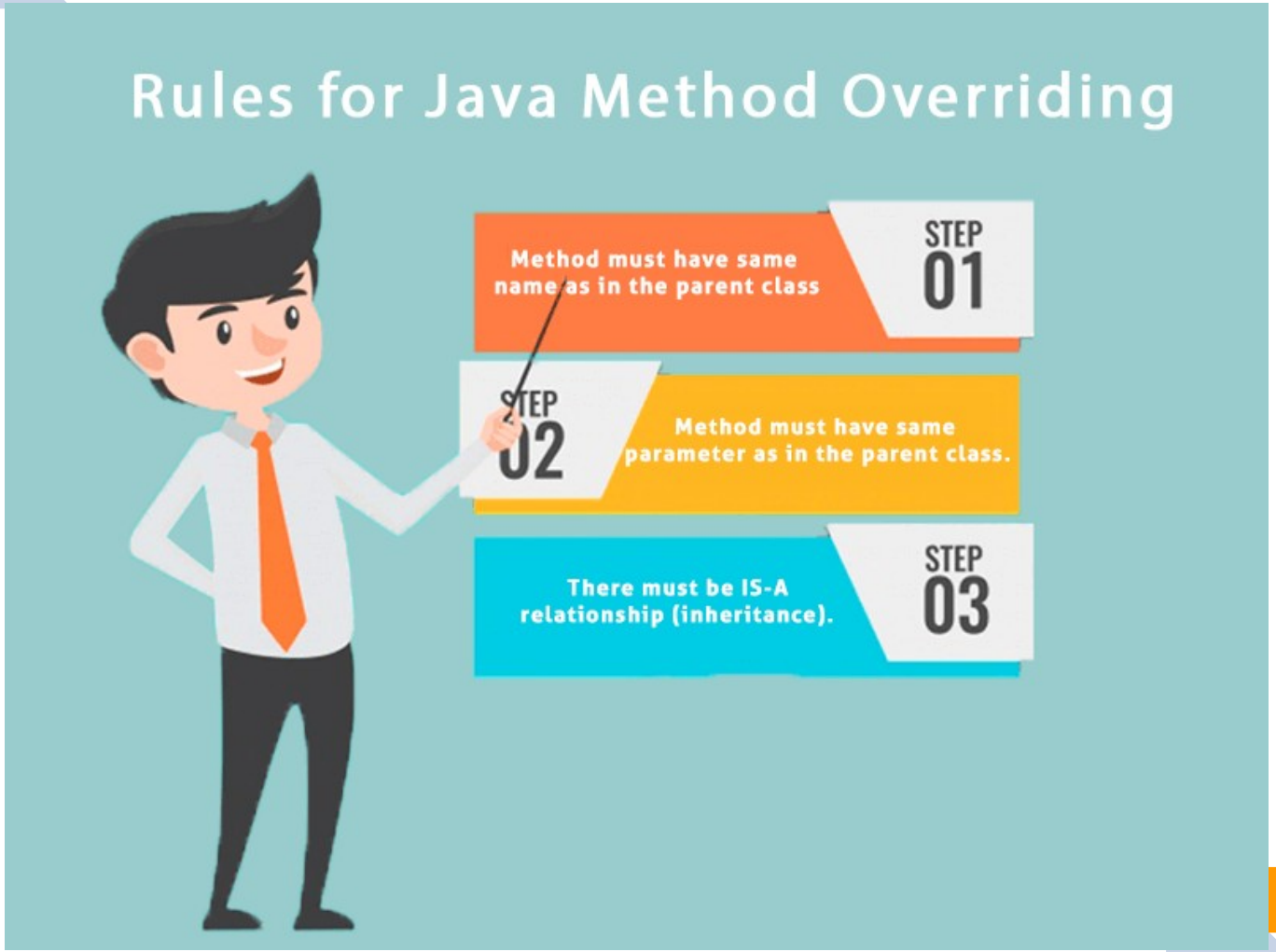
If subclass (child class) has the same method as declared in the parent class, it is known as method overriding.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

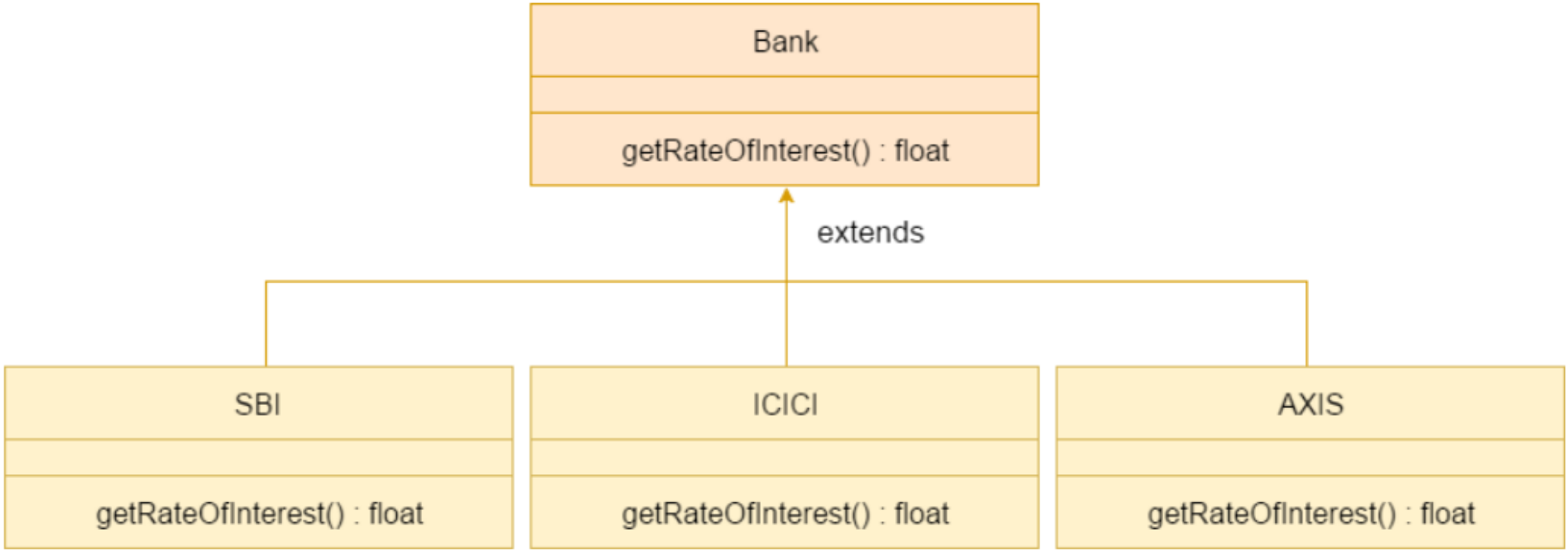
- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Method Overriding



A real example of Java Method Overriding

Consider a scenario where Bank is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



Method Overriding

```
class Bank{
int getRateOfInterest(){return 0;}
}
//Creating child classes.
class SBI extends Bank{
int getRateOfInterest(){return 8;}
}

class ICICI extends Bank{
int getRateOfInterest(){return 7;}
}
class AXIS extends Bank{
int getRateOfInterest(){return 9;}
}
```

```
//Test class to create objects and call the methods
class Test2{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
}
}
```

Output:

```
SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9
```

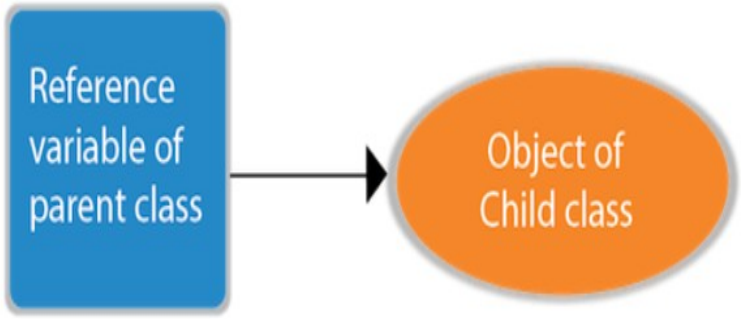
Differences between Method overloading and Method overriding

No.	Method Overloading	Method Overriding
1)	Method overloading is used *to increase the readability* of the program.	Method overriding is used *to provide the specific implementation* of the method that is already provided by its super class.
2)	Method overloading is performed *within class*.	Method overriding occurs *in two classes* that have IS-A (inheritance) relationship.
3)	In case of method overloading, *parameter must be different*.	In case of method overriding, *parameter must be same*.
4)	Method overloading is the example of *compile time polymorphism*.	Method overriding is the example of *run time polymorphism*.
5)	In java, method overloading can't be performed by changing return type of the method only. *Return type can be same or different* in method overloading. But you must have to change the parameter.	*Return type must be same or covariant* in method overriding.

Runtime Polymorphism / Dynamic Method Dispatch

- Runtime polymorphism or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time.
- In this process, an overridden method is called through the *reference variable of a superclass*.
- The determination of the method to be called is based on the object being referred to by the reference variable.
- Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.
- If the reference variable of Parent class refers to the object of Child class, it is known as upcasting.

Upcasting




```
class A{}
class B extends A{}

A a=new B();//upcasting
```

For upcasting, we can use the reference variable of class type or an interface type.

```
interface I{}
class A{}
class B extends A implements I{}
```

Here, the relationship of B class would be:

B IS-A A
B IS-A I
B IS-A Object

Runtime Polymorphism / Dynamic Method Dispatch

```
class A
{
    void m1()
    {System.out.println("Inside A's m1 method");}
}

class B extends A
{
    // overriding m1()
    void m1()
    {System.out.println("Inside B's m1 method");}
}

class C extends A
{
    // overriding m1()
    void m1()
    {System.out.println("Inside C's m1 method");}
}
```

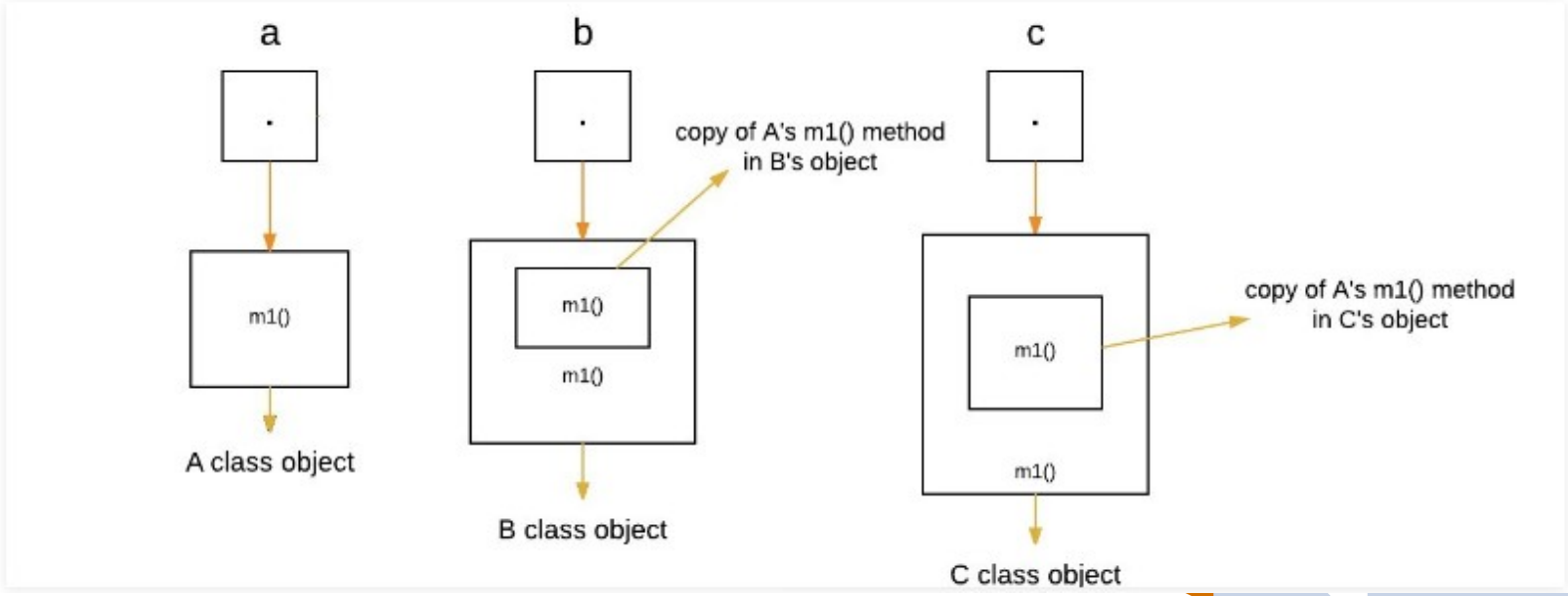
```
// Driver class
class Dispatch
{
    public static void main(String args[])
    {
        // objects of type A, B
        A a = new A();
        B b = new B();

        // obtain a reference of type A
        A ref;
        // ref refers to an A object
        ref = a;
        // calling A's version of m1()
        ref.m1();

        // now ref refers to a B object
        ref = b;
        // calling B's version of m1()
        ref.m1();

        // now ref refers to a C object
        ref = new C();
        // calling C's version of m1()
        ref.m1();
    }
}
```

Explanation :

The above program creates one superclass called A and it's two subclasses B and C. These subclasses overrides m1( ) method.

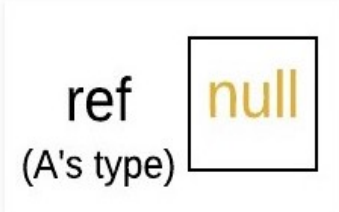
1. Inside the main() method in Dispatch class, initially objects of type A, B, and C are declared.

```
A a = new A(); // object of type A
B b = new B(); // object of type B
C c = new C(); // object of type C
```

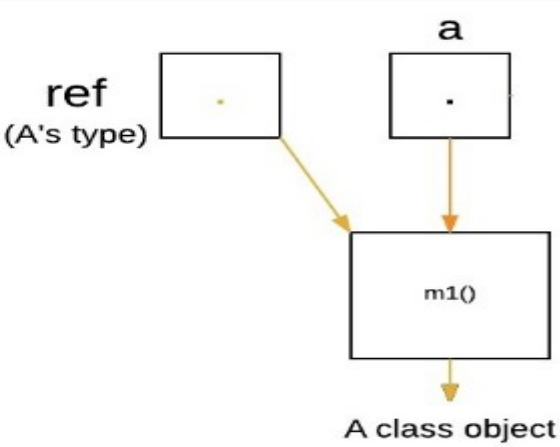


2. Now a reference of type A, called ref, is also declared, initially it will point to null.

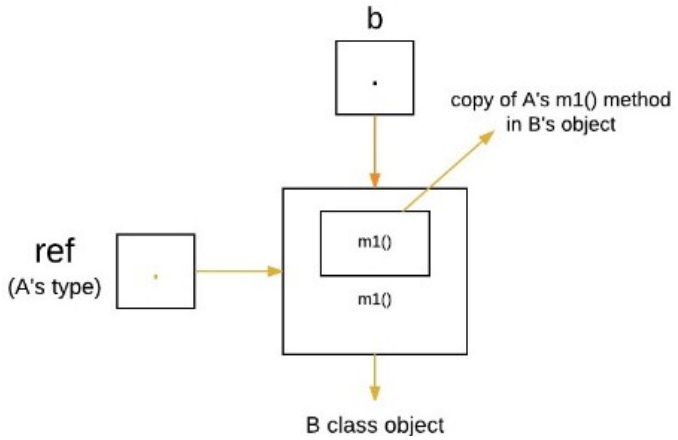
```
A ref; // obtain a reference of type A
```



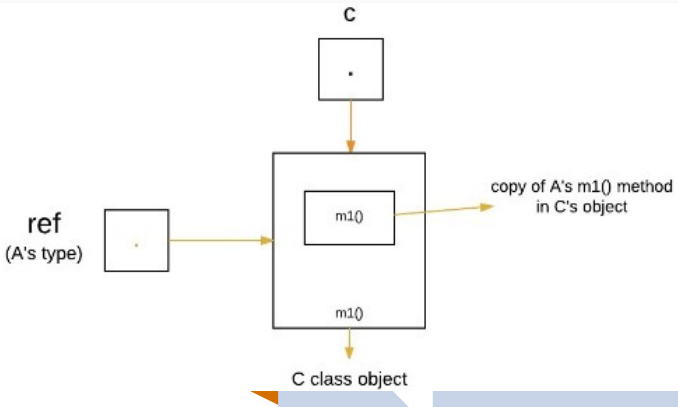

```
ref = a; // r refers to an A object
ref.m1(); // calling A's version of m1()
```




```
ref = b; // now r refers to a B object
ref.m1(); // calling B's version of m1()
```




```
ref = c; // now r refers to a C object
ref.m1(); // calling C's version of m1()
```



Runtime Polymorphism with Data Member

A method is overridden, not the data members, so runtime polymorphism can't be achieved by data members.

```
class Bike{
 int speedlimit=90;
}
class Honda3 extends Bike{
 int speedlimit=150;

 public static void main(String args[]){
  Bike obj=new Honda3();
  System.out.println(obj.speedlimit);
}
}
```

Output:
90

Abstraction

- Abstraction is a process of hiding the implementation details and showing only functionality to the user.
- It shows only essential things to the user and hides the internal details,
- for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

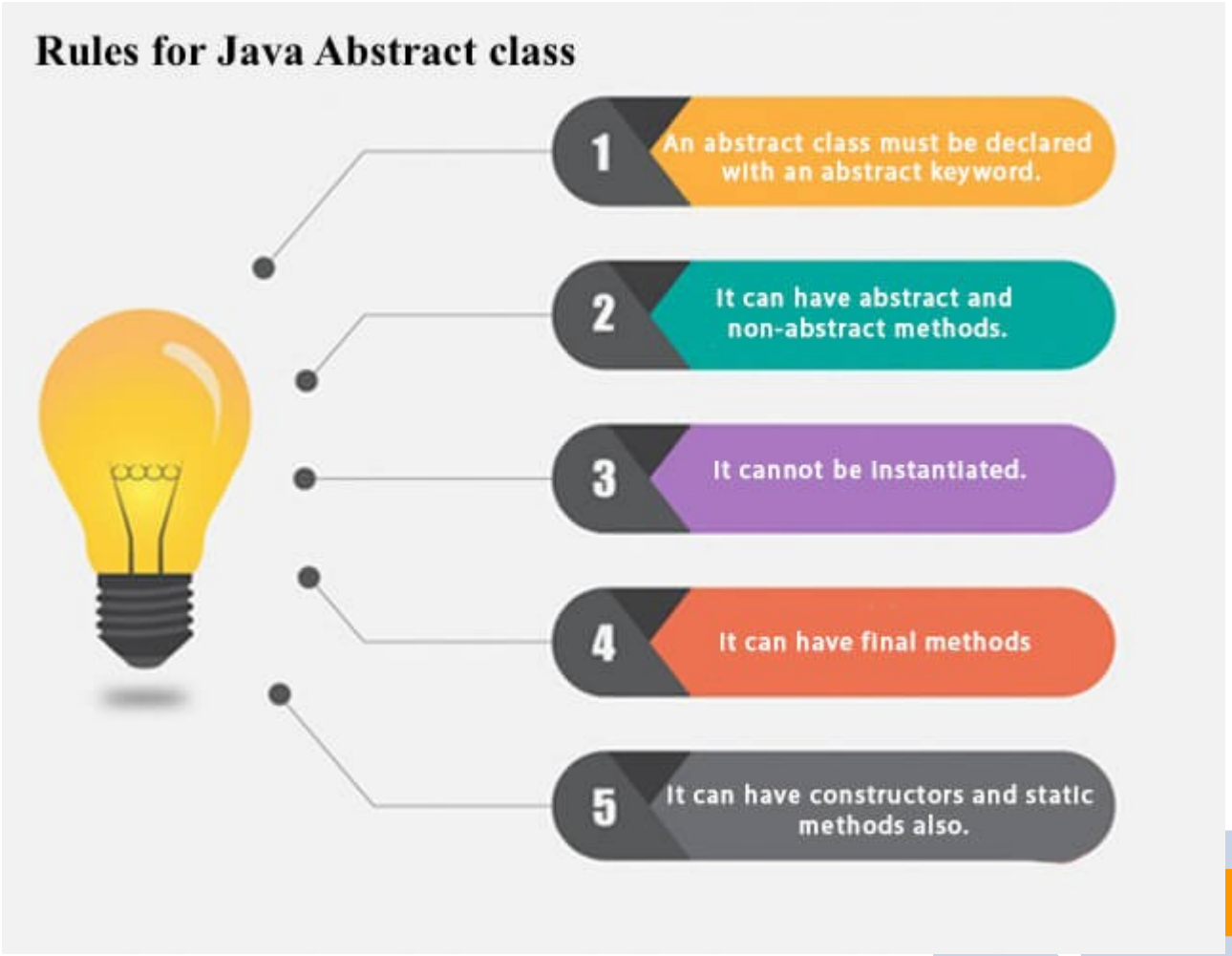
There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class

A class which is declared as abstract is known as an abstract class.

A method which is declared as abstract and does not have implementation is known as an abstract method



Abstract class

```
abstract class Bank{
abstract int getRateOfInterest();
}
class SBI extends Bank{
int getRateOfInterest(){return 7;}
}
class PNB extends Bank{
int getRateOfInterest(){return 8;}
}

class TestBank{
public static void main(String args[]){
Bank b;
b=new SBI();
System.out.println("Rate of Interest is in SBI: "+b.getRateOfInterest()+" %");
b=new PNB();
System.out.println("Rate of Interest is in PNB: "+b.getRateOfInterest()+" %");
}}
```

Output:

```
Rate of Interest is in SBI: 7 %
Rate of Interest is in PNB: 8 %
```

Abstract class having constructor, data member and methods

```
//Example of an abstract class that has abstract and non-abstract methods
 abstract class Bike{
   Bike(){System.out.println("bike is created");}
   abstract void run();
   void changeGear(){System.out.println("gear changed");}// non-abstract
 }

//Creating a Child class which inherits Abstract class
 class Honda extends Bike{
 void run(){System.out.println("running safely..");}
 }

//Creating a Test class which calls abstract and non-abstract methods
 class Main{
 public static void main(String args[]){
  Bike obj = new Honda();
  obj.run();
  obj.changeGear();
 }
}
```

Output:

```
bike is created
running safely..
gear changed
```

Rule: If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract.

```
abstract class Bike{
  Bike(){System.out.println("bike is created");}
  abstract void run();
  void changeGear(){System.out.println("gear changed");}// non-abstract
}

class Honda extends Bike{}

class Main{
public static void main(String args[]){
 Bike obj = new Honda();
 obj.run();
 obj.changeGear();
 }
}
```

Output:

```
Main.java:9: error: Honda is not abstract and does not override abstract method run() i
     class Honda extends Bike{
     ^
1 error
```

Solution:

```
abstract class Bike{
  Bike(){System.out.println("bike is created");}
  abstract void run();
  void changeGear(){System.out.println("gear changed");},
}

abstract class Honda extends Bike{}

class Honda2 extends Honda{
 void run(){System.out.println("running safely..");}
 }

class Main{
public static void main(String args[]){
 Bike obj = new Honda2();
 obj.run();
 obj.changeGear();
 }
}
```

Output:

```
bike is created
running safely..
gear changed
```

Unit 2

Packages & Interfaces

Interface

- An interface in Java is a blueprint of a class.
- It has static constants and abstract methods.
- The interface in Java is a mechanism to achieve abstraction and multiple inheritance in Java.
- Java Interface also represents the IS-A relationship.
- It cannot be instantiated just like the abstract class.
- Since Java 8, we can have default and static methods in an interface.
- Since Java 9, we can have private methods in an interface.

How to declare an interface?

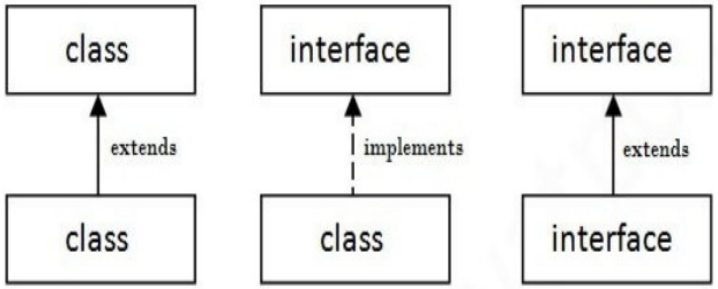
- An interface is declared by using the interface keyword.
- It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.
- The Java compiler adds public and abstract keywords before the interface method. Moreover, it adds public, static and final keywords before data members.
- A class that implements an interface must implement all the methods declared in the interface.

How to declare an interface?

Syntax:

```
interface <interface_name>
{

    // declare constant fields
    // declare methods that abstract
    // by default.
 }
```

The relationship between classes and interfaces



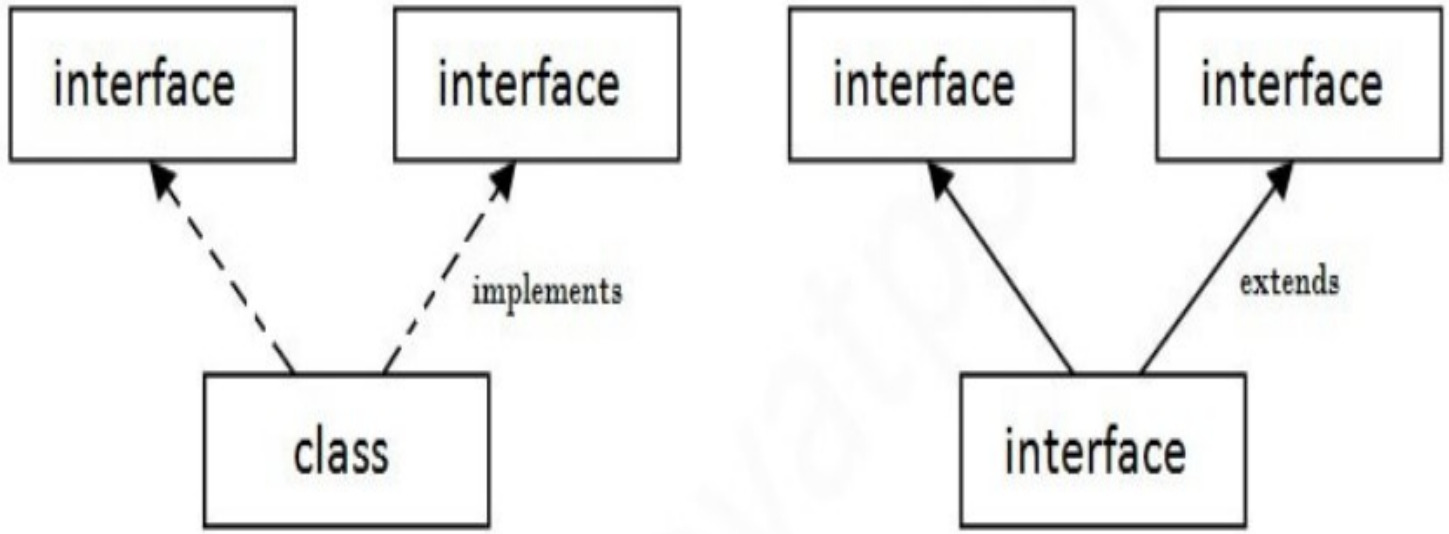

```
interface Bank{
float rateOfInterest();
}
class SBI implements Bank{
public float rateOfInterest(){return 9.15f;}
}
class PNB implements Bank{
public float rateOfInterest(){return 9.7f;}
}
class Main{
public static void main(String[] args){
Bank b;
b=new SBI();
System.out.println("SBI ROI: "+b.rateOfInterest());

b=new PNB();
System.out.println("PNB ROI: "+b.rateOfInterest());
}}
```

Output:

```
SBI ROI: 9.15
PNB ROI: 9.7
```

Multiple Inheritance by Interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance by Interface

```
interface Printable{
void print();
}
interface Showable{
void show();
}
class Inheritance implements Printable,Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
Inheritance obj = new Inheritance();
obj.print();
obj.show();
 }
}
```

Output:

```
Hello
Welcome
```

Interface inheritance

- A class implements an interface, but one interface can extend another interface also.
- Since Java 8, we can have method body in interface.
- But we need to make it default method.

Interface inheritance

```
interface Printable{
final String s="Nitte";
void print();
}

interface Showable extends Printable{
void show();
default void msg(){System.out.println("default method");}
}

class Inheritance implements Showable{
public void print(){System.out.println("Hello "+s);}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
Inheritance obj = new Inheritance();
obj.print();
obj.show();
obj.msg();
 }
}
```

Output:

```
Hello Nitte
Welcome
default method
```

Interface inheritance

- The abstract class can also be used to provide some implementation of the interface.
- In such case, the end user may not be forced to override all the methods of the interface.

```
interface A{
void a();
void b();
void c();
void d();
}

abstract class B implements A{
public void c(){System.out.println("I am c");}
}

class M extends B{
public void a(){System.out.println("I am a");}
public void b(){System.out.println("I am b");}
public void d(){System.out.println("I am d");}
}

class Test5{
public static void main(String args[]){
A a=new M();
a.a();
a.b();
a.c();
a.d();
}}
```

Output:

```
I am a
I am b
I am c
I am d
```

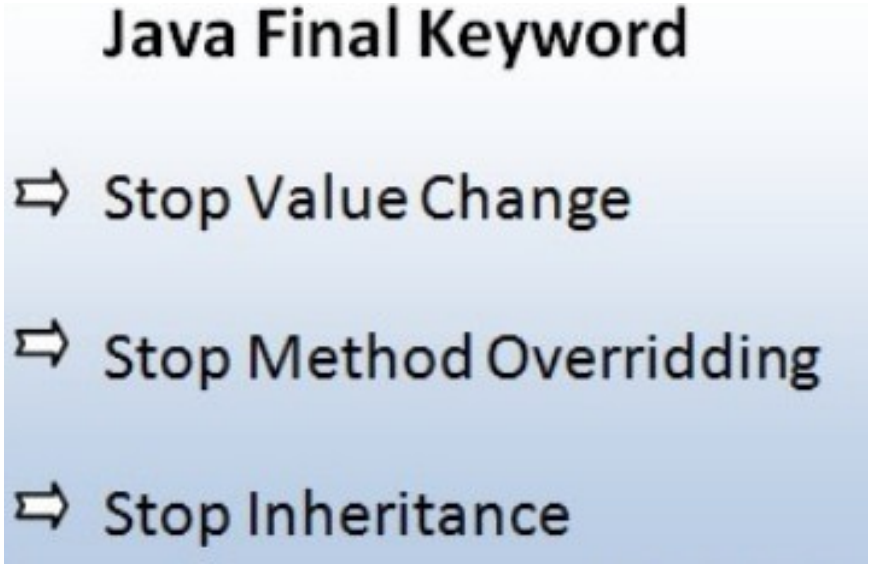
Differences between Abstract class and Interface

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods** also.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.

Differences between Abstract class and Interface

Abstract class	Interface
5) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.
6) An **abstract class** can extend another Java class and implement multiple Java interfaces.	An **interface** can extend another Java interface only.
7) An **abstract class** can be extended using keyword "extends".	An **interface** can be implemented using keyword "implements".
8) A Java **abstract class** can have class members like private, protected, etc.	Members of a Java interface are public by default.

Final Keyword



Example

1) Java final variable

```
class Bike{
final int speedlimit=90;//final variable
void run(){
 speedlimit=400;
}
public static void main(String args[]){
Bike obj=new  Bike();
obj.run();
}
}//end of class
```

Output:

```
Main.java:4: error: cannot assign a value to final variable speedlimit
      speedlimit=400;
      ^
1 error
```

Example

2) Java final method

```
  class Bike{
  final void run(){System.out.println("running");}
  }

class Honda extends Bike{
   void run(){System.out.println("running safely with 100kmph");}

   public static void main(String args[]){
   Honda honda= new Honda();
   honda.run();
   }
}
```

Output:

```
Main.java:6: error: run() in Honda cannot override run() in Bike
       void run(){System.out.println("running safely with 100kmph");}
            ^
  overridden method is final
1 error
```

Example

3) Java final class

```
final class Bike{}

class Honda1 extends Bike{
  void run(){System.out.println("running safely with 100kmph");}

  public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

Output:

```
Main.java:3: error: cannot inherit from final Bike
    class Honda1 extends Bike{
                         ^
1 error
```

Final Keyword

Q) Is final method inherited?

Ans) Yes, final method is inherited but you cannot override it.

```
  class Bike{
  final void run(){System.out.println("running...");}
  }
class Honda2 extends Bike{
   public static void main(String args[]){
    new Honda2().run();
   }
}
```

Output:

```
running...
```

Final Keyword

Q) What is final parameter?

If you declare any parameter as final, you cannot change the value of it.

```
class Bike11{
 int cube(final int n){
  n=n+2;//can't be changed as n is final
  n*n*n;
 }
 public static void main(String args[]){
   Bike11 b=new Bike11();
   b.cube(5);
 }
}
```

Output:

```
Main.java:4: error: not a statement
       n*n*n;
          ^
1 error
```

Aggregation

- If a class have an entity reference, it is known as Aggregation.
- Aggregation represents HAS-A relationship.
- Consider a situation, Employee object contains many informations such as id, name, emailId etc.
- It contains one more object named address, which contains its own informations such as city, state, country, zipcode etc.

Aggregation

```
class Employee{

int id;

String name;

Address address;//Address is a class

...

}
```

Here, Employee has an entity reference address, so relationship is Employee HAS-A address.

Aggregation

Example

```
class Address {
String city,state,country;

public Address(String city, String state, String country){
    this.city = city;
    this.state = state;
    this.country = country;
}

}
public class Emp {
int id;
String name;
Address address;

public Emp(int id, String name,Address address) {
    this.id = id;
    this.name = name;
    this.address=address;}
```

Example

Aggregation

```
void display(){
System.out.println(id+" "+name);
System.out.println(address.city+" "+address.state+" "+address.country);}

public static void main(String[] args) {
Address address1=new Address("Nitte","Karnataka","India");
Address address2=new Address("Chennai","Tamilnad","India");
Address address3=new Address("Kasargod","Kerala","India");

Emp e=new Emp(111,"Ram",address1);
Emp e2=new Emp(112,"Sham",address2);
Emp e3=new Emp(113,"Bhim",address3);
e.display();
e2.display();
e3.display();
}
}
```

Output:

```
111 Ram
Nitte Karnataka India
112 Sham
Chennai Tamilnad India
113 Bhim
Kasargod Kerala India
```

Example

Aggregation

```
void display(){
System.out.println(id+" "+name);
System.out.println(address.city+" "+address.state+" "+address.country);}

public static void main(String[] args) {
Address address1=new Address("Nitte","Karnataka","India");
Address address2=new Address("Chennai","Tamilnad","India");
Address address3=new Address("Kasargod","Kerala","India");

Emp e=new Emp(111,"Ram",address1);
Emp e2=new Emp(112,"Sham",address2);
Emp e3=new Emp(113,"Bhim",address3);
e.display();
e2.display();
e3.display();
}
}
```

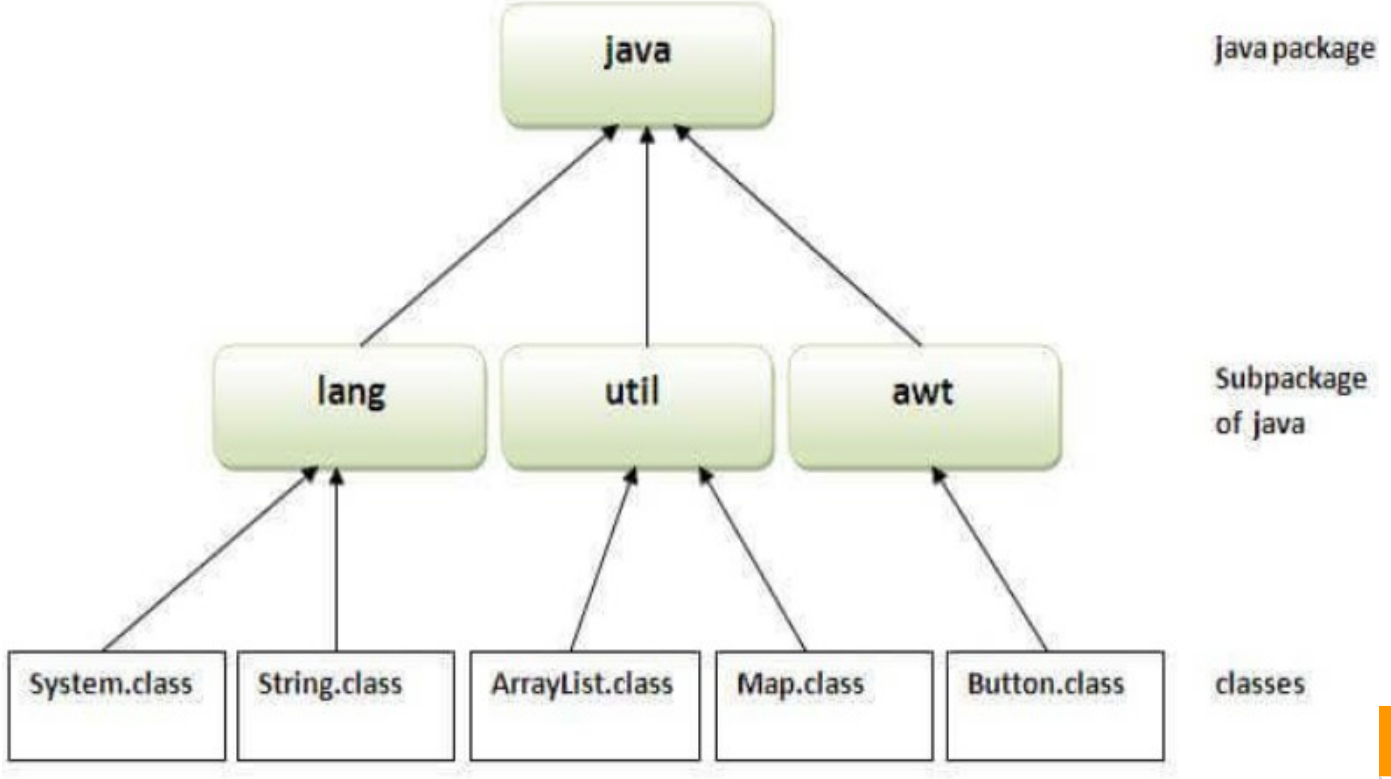
Output:

```
111 Ram
Nitte Karnataka India
112 Sham
Chennai Tamilnad India
113 Bhim
Kasargod Kerala India
```

Packages

A java package is a group of similar types of classes, interfaces and sub-packages.

Package in java can be categorized in two form, built-in package and user-defined package.



Package

Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces.

Packages are used for:

- Preventing naming conflicts. For example there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
- Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
- Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation (or data-hiding).

Simple example of creating Java package

```
package mypack;
public class Main{
 public static void main(String args[]){
    System.out.println("Welcome to package");
   }
}
```

To Compile and Run java package

If you are not using any IDE, you need to follow the syntax given below:

To Compile: javac -d . Simple.java
To Run: java mypack.Simple

- The -d switch specifies the destination where to put the generated class file.
- You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc.
- If you want to keep the package within the same directory, you can use . (dot).

How to access package from another package?

1) Using packagename.*

- If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

- The import keyword is used to make the classes and interface of another package accessible to the current package.

How to access package from another package?

```
//save by A.java
package pack;
public class A{
  public void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.*;

class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

How to access package from another package?

2) Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

```
//save by A.java

package pack;
public class A{
  public void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.A;

class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

How to access package from another package?

3) Using fully qualified name

- If you use fully qualified name then only declared class of this package will be accessible.
- Now there is no need to import.
- But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

How to access package from another package?

```
//save by A.java
package pack;
public class A{
  public void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
class B{
  public static void main(String args[]){
   pack.A obj = new pack.A();//using fully qualified name
   obj.msg();
  }
}
```

Sub-package

- Package inside the package is called the sub-package. It should be created to categorize the package further.
- For example, Sun Microsystem has defined a package named java that contains many classes like System, String, Reader, Writer, Socket etc.
- These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on.
- So, Sun has sub-categorized the java package into sub-packages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

Sub-package

How to create and use sub packages

```
package mypack.testpack;

class MySubPackageProgram {
   public static void main(String args [])
{
   System.out.println("My sub package
program");
    }
 }
```

How to import Sub packages

```
// To import all classes of a sub package
import packagename.subpackagename.*;

// To import specific class of a sub package
import packagename.subpackagename.classname;
```

Example

```
import mypack.testpack.*;
import mypack.testpack.MySubPackageProgram;
```

Package – Interface Example

```java
//Arithmetic.java
package intrface;
public interface Arithmetic
{
void add();
void sub();
}
```

```java
//Addtion.java
package addpack;
import intrface.Arithmetic;
public abstract class addition implements intrface.Arithmetic
{
public void add()
{
int a=1,b=2,s=0;
s=a+b;
System.out.println("Sum:"+s);
}
}
```

Package – Interface Example

```java
//Subtraction.java
package addpack;
import intrface.Arithmetic;
public class subtraction extends addpack.addition
{
public void sub()
{
int a=1,b=2,s=0;
s=a-b;
System.out.println("Difference:"+s);
}
}
```

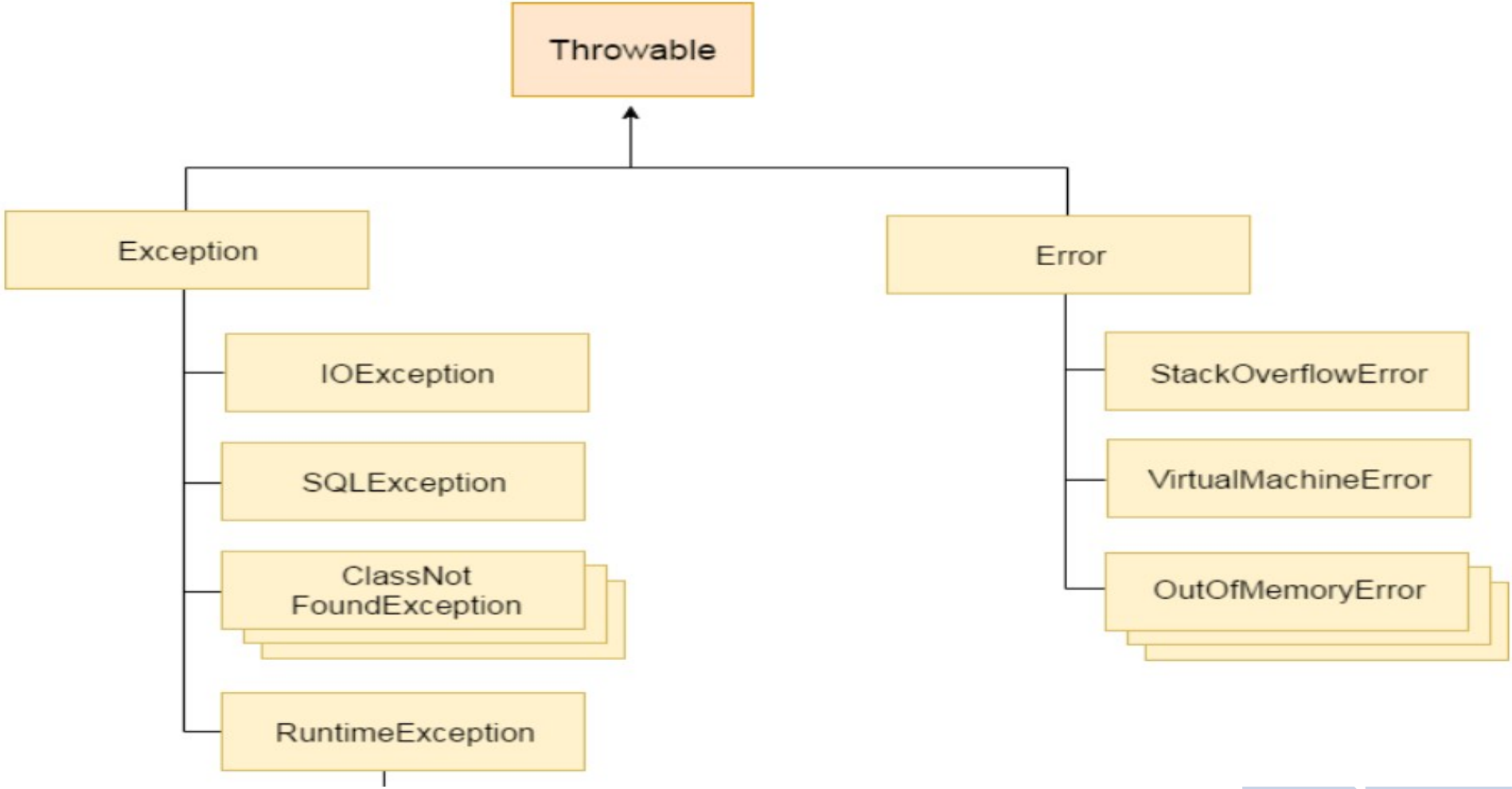
```java
//ExamInterface.java
import addpack.subtraction;
class ExamInterface
{
public static void main(String args[])
{
subtraction s=new subtraction();
s.add();
s.sub();
}
}
```

Unit 2
Exception Handling

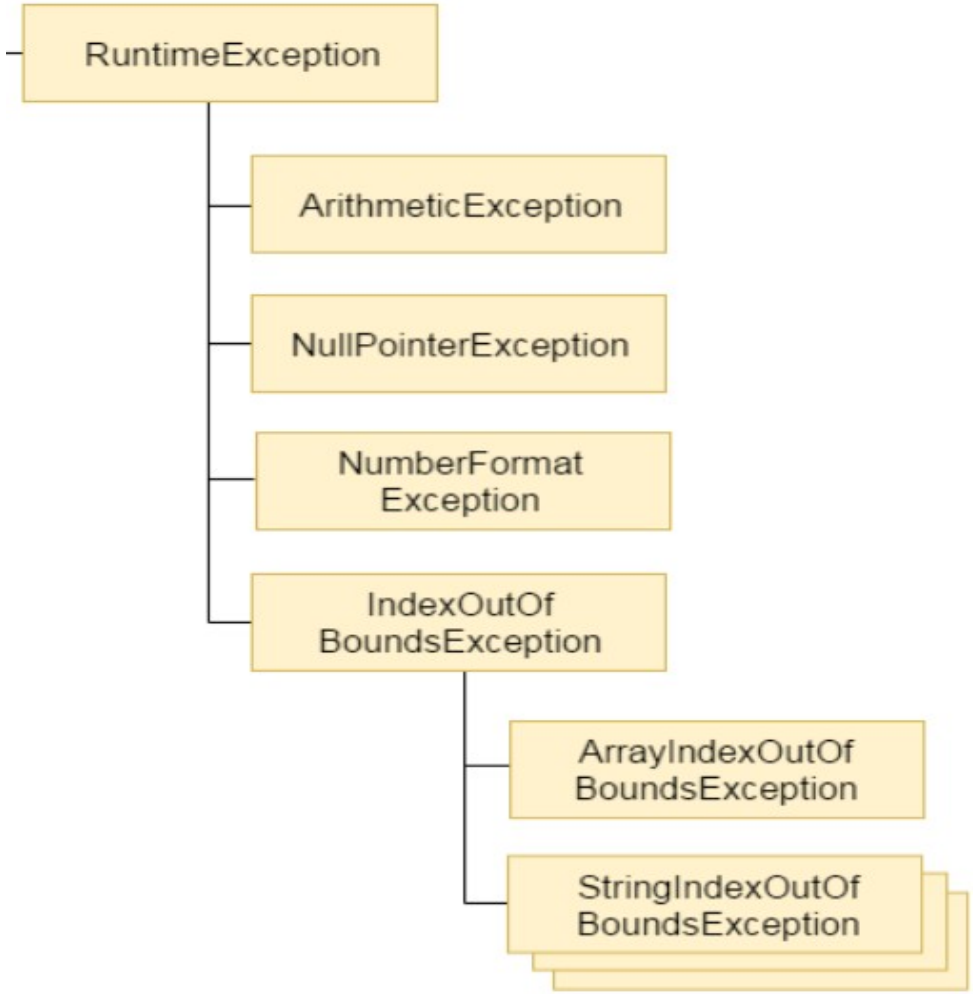
Exception Handling

- The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.
- Exception is an abnormal condition.
- Exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.
- Suppose there are 10 statements in your program and there occurs an exception at statement 5, the rest of the code will not be executed i.e. statement 6 to 10 will not be executed. If we perform exception handling, the rest of the statement will be executed. That is why we use exception handling in Java.

Hierarchy of Java Exception classes



Hierarchy of Java Exception classes



Types of Java Exceptions

Types of Java Exceptions

1) Checked Exception

The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes which inherit RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Common Scenarios of Java Exceptions

1) int a=50/0;//ArithmeticException

2) String s=null;
   System.out.println(s.length());//NullPointerException

3) String s="abc";
   int i=Integer.parseInt(s);//NumberFormatException

4) int a[]=new int[5];
   a[10]=50; //ArrayIndexOutOfBoundsException

Java Exception Keywords

There are 5 keywords which are used in handling exceptions in Java.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

Java try-catch block

- Java try block is used to enclose the code that might throw an exception. It must be used within the method.
- If an exception occurs at the particular statement of try block, the rest of the block code will not execute. So, it is recommended not to keeping the code in try block that will not throw an exception.
- Java try block must be followed by either catch or finally block.

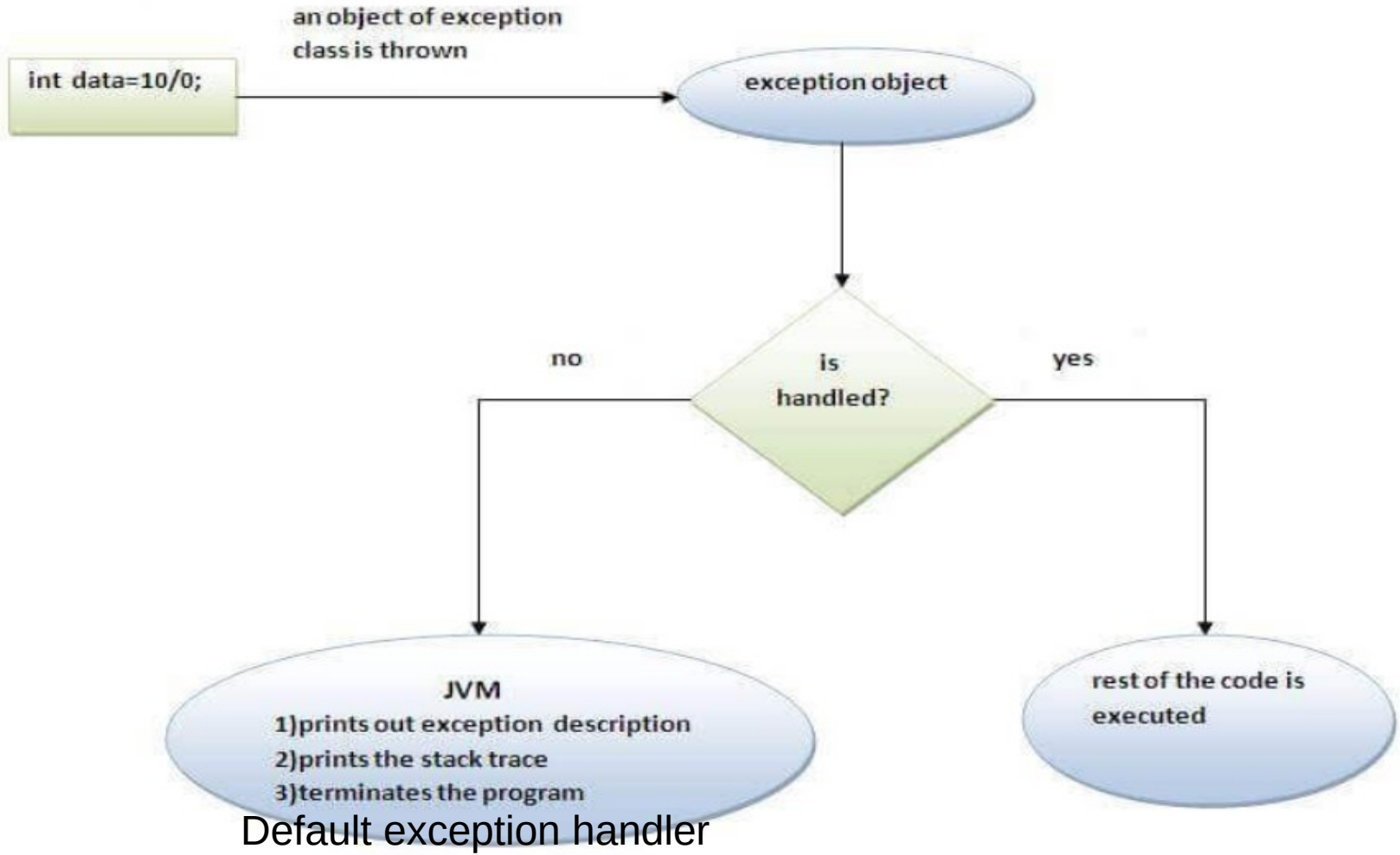
Syntax of Java try-catch

```
try{
 //code that may throw an exception
}catch(Exception_class_Name ref){}
```

Syntax of try-finally block

```
try{
 //code that may throw an exception
}finally{}
```

Internal working of java try-catch block



Default exception handler

Java try-catch block

Problem without exception handling

```
public class TryCatchExample{
  public static void main(String[] args){
  int data=50/0; //may throw exception
  System.out.println("rest of the code");
  }
}
```

Output:
Exception in thread "main"
java.lang.ArithmeticException: /
by zero

The rest of the code is not executed (in such case, the rest of the code statement is not printed).

Solution by exception handling

```
public class Main{
 public static void main(String[] args) {
 try
 {
  int data=50/0; //may throw exception
 }
 //handling the exception
 catch(ArithmeticException e)
 {
  System.out.println(e);
 }
 System.out.println("Rest of the code");
 }
}
```

Output:

```
java.lang.ArithmeticException: / by zero
Rest of the code
```

Java try-catch block

Resolving the exception in a catch block.

```
public class Main{
 public static void main(String[] args) {
 int i=50,j=0;
 try
 {
   i=i/j; //may throw exception
  // if exception occurs, the remaining statement will not exceute
  System.out.println("Rest of the code");
 }
// handling the exception
catch(Exception e)
{
 System.out.println(i/(j+2));
}
}
}
```

Output:

25

Java catch multiple exceptions

- A try block can be followed by one or more catch blocks.
- Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.
- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Java catch multiple exceptions

```
public class Main {
public static void main(String[] args) {
try{
    int a[]=new int[5];
    a[5]=30/0;
   }
   catch(ArrayIndexOutOfBoundsException e)
   {System.out.println("ArrayIndexOutOfBounds Exception occurs");}
   catch(ArithmeticException e)
   {System.out.println("Arithmetic Exception occurs");}
   catch(Exception e)
   {System.out.println("Parent Exception occurs");}

   System.out.println("Rest of the code");
 }
}
```

Output:

```
Arithmetic Exception occurs
Rest of the code
```

Java catch multiple exceptions

```
class Main{
  public static void main(String args[]){
   try{
    int a[]=new int[5];
    a[5]=30/0;
   }
   catch(Exception e){System.out.println("common task completed");}
   catch(ArithmeticException e){System.out.println("task1 is completed");}
   catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
   System.out.println("rest of the code...");
 }
}
```

Output:

Compile-time error

```
Main.java:8: error: exception ArithmeticException has already been caught
       catch(ArithmeticException e){System.out.println("task1 is completed");}
       ^
Main.java:9: error: exception ArrayIndexOutOfBoundsException has already been caught
       catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
       ^
2 errors
```

Java Nested try block

- The try block within a try block is known as nested try block in java.
- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error.
- In such cases, exception handlers have to be nested.

Syntax:

```
....
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
}
....
```

Java Nested trv block

```
class Main{
public static void main(String args[]){
 try{
   try{
    System.out.println("going to divide");
    int b =39/0;
   }catch(ArithmeticException e){System.out.println(e);}

   try{
   int a[]=new int[5];
   a[5]=4;
   }catch(ArrayIndexOutOfBoundsException e){System.out.println(e);}

   System.out.println("other statement");
 }catch(Exception e){System.out.println("handeled");}

 System.out.println("normal flow..");
 }
}
```

Output:

```
going to divide
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
other statement
normal flow..
```

Java finally block

Java finally block is always executed whether exception is handled or not.

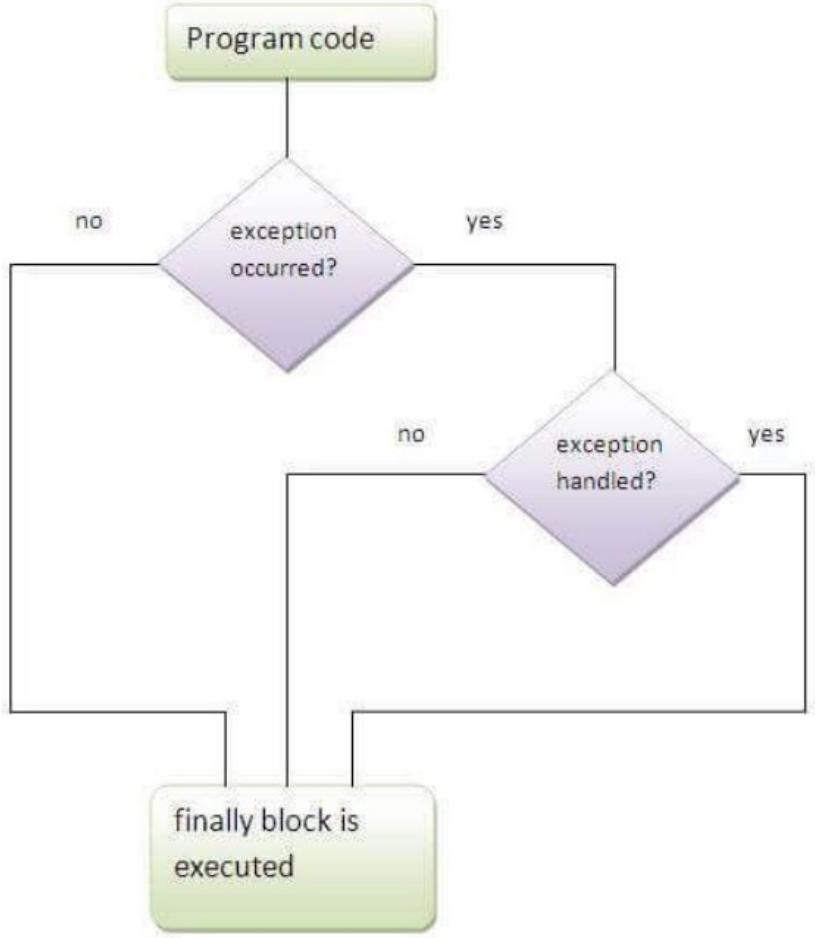
Java finally block follows try or catch block.

If you don't handle exception, before terminating the program, JVM executes finally block(if any).

Finally block in java can be used to put "cleanup" code such as closing a file, closing connection etc.

For each try block there can be zero or more catch blocks, but only one finally block.

The finally block will not be executed if program exits(either by calling System.exit() or by causing a fatal error that causes the process to abort).



Different cases where java finally block can be used

Case 1: where exception doesn't occur.

```
class Main{
  public static void main(String args[]){
  try{
   int data=25/5;
   System.out.println(data);
  }
  catch(NullPointerException e){System.out.println(e);}
  finally{System.out.println("finally block is always executed");}
  System.out.println("rest of the code...");
  }
}
```

Output:

```
5
finally block is always executed
rest of the code...
```

Different cases where java finally block can be used

Case 2: where exception occurs and not handled.

```
class TestFinallyBlock1{
  public static void main(String args[]){
  try{
   int data=25/0;
   System.out.println(data);
  }
  catch(NullPointerException e){System.out.println(e);}
  finally{System.out.println("finally block is always executed");}
  System.out.println("rest of the code...");
  }
}
```

Output:

```
finally block is always executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestFinallyBlock1.main(TestFinallyBlock1.java:6)
```

Different cases where java finally block can be used

Case 3: where exception occurs and handled.

```
public class Main{
  public static void main(String args[]){
  try{
   int data=25/0;
   System.out.println(data);
  }
  catch(ArithmeticException e){System.out.println(e);}
  finally{System.out.println("finally block is always executed");}
  System.out.println("rest of the code...");
  }
}
```

Output:

```
java.lang.ArithmeticException: / by zero
finally block is always executed
rest of the code...
```

Java throw Keyword

- The Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or uncheked exception in java by throw keyword.
- The throw keyword is mainly used to throw custom exception.

Syntax: throw exception;

Example: throw new IOException("sorry device error);

Java throw exception

```
 public class Main{
static void validate(int age){
  if(age<18)
   throw new ArithmeticException("not valid");
  else
   System.out.println("welcome to vote");
}
public static void main(String args[]){
   validate(13);
   System.out.println("rest of the code...");
}
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: not valid
        at Main.validate(Main.java:4)
        at Main.main(Main.java:9)
```

Stack Trace

Java throws keyword

- The throws keyword indicates what exception type may be thrown by a method.
- The Java throws keyword is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Java throws keyword

- Exception Handling is mainly used to handle the checked exceptions.

  Bcoz....

  1. unchecked Exception: under your control so correct your code.

  2. error: beyond your control e.g. you are unable to do anything if there occurs

     VirtualMachineError or StackOverflowError.

- If there occurs any unchecked exception such as NullPointerException, it is programmers fault that he is not performing check up before the code being used.

Java throws keyword

Syntax of java throws

return_type method_name() throws exception_class_name{
//method code
}

```
public class Main {
  static void checkAge(int age) throws ArithmeticException {
    if (age < 18) {
      throw new ArithmeticException("Access denied - You must be at least 18 years old.");
    }
    else {
      System.out.println("Access granted - You are old enough!");
    }
  }

  public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
  }
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: Access denied - You must be at
    least 18 years old.at Main.checkAge(Main.java:4)at Main.main(Main.java:12)
```

Differences between throw and throws

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

Java Custom Exception

- If you are creating your own Exception that is known as custom exception or user-defined exception.
- Java custom exceptions are used to customize the exception according to user need.
- Java provides us facility to create our own exceptions which are basically derived classes of Exception.

Java Custom Exception

Exception class with super()

```
class InvalidAgeException extends Exception
{
 InvalidAgeException(String s)
 {
  super(s);
 }
}
```

Exception class without super()

```
class InvalidAgeException extends Exception
{

}
```

We pass the string to the constructor of the super class- Exception which is obtained using “getMessage()” function on the object created.

Java Custom Exception

```
class InvalidAgeException extends Exception
{
 InvalidAgeException(String s)
 {
  super(s);
 }
}

   class Main{

   static void validate(int age)throws InvalidAgeException{
     if(age<18)
      throw new InvalidAgeException("not valid");
     else
      System.out.println("welcome to vote");
   }

   public static void main(String args[]){
      try{
      validate(13);
      }catch(Exception m){System.out.println("Exception occured: "+m);}

      System.out.println("rest of the code...");
  }
}
```

Output:

```
Exception occured: InvalidAgeException: not valid
rest of the code...
```

Java Custom Exception

```
class InvalidAgeException extends Exception
{

}

   class Main{

   static void validate(int age)throws InvalidAgeException{
     if(age<18)
      throw new InvalidAgeException();
     else
      System.out.println("welcome to vote");
   }

   public static void main(String args[]){
      try{
      validate(13);
      }catch(Exception m){System.out.println("Exception occured: "+m);}

      System.out.println("rest of the code...");
  }
}
```

Output:

```
Exception occured: InvalidAgeException
rest of the code...
```